

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

LABORATORY OBSERVATION & RECORD

Basic Information

Student Name: M. Surya Prabhath

Roll No: A24126552094

Branch & Sec: CSM-B

Date of Execution: 21/09/2026

Course Name: Full Stack Web Development

Course Code: 23CM4121

Task No: 05

Title: Interactive Product Calculator using HTML, CSS, and JavaScript

Objective / Aim

To design and build an interactive Product Calculator web application using HTML, CSS, and JavaScript. The application dynamically calculates the total cost whenever the product name, price, or quantity is updated, providing immediate visual feedback and error handling without requiring a page refresh.

System Requirements

Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent
- Memory: 8 GB RAM
- Storage: Standard SSD / HDD

Software Requirements

- Operating System: Windows 11 / Linux / macOS
- Code Editor: Visual Studio Code
- Web Browser: Google Chrome / Mozilla Firefox / Microsoft Edge

How the Application Works (Project Flow)

The Interactive Product Calculator is designed to give users real-time feedback as they type, creating a seamless experience without needing form submission buttons or page reloads.

Initial Page Load

When the web page opens, the browser reads the structure from index.html and styles it using style.css. The JavaScript file (script.js) runs automatically, grabbing the input fields (Name, Price, Quantity) and display elements (Selected Product Name, Total Amount) from the DOM. It performs an initial check so default values are immediately calculated and displayed.

Listening for User Interaction

Event listeners are attached to the input boxes for the input event. Unlike traditional forms that wait for a submit button, this event fires instantly every time the user types a single character, changes a number, or uses arrow toggles inside an input field.

Input Extraction and Parsing

Whenever an event is triggered, the **calculateTotal()** function runs:

- It extracts the product name and strips extra whitespace **using .trim()**.
- It converts the price input into a decimal number using **parseFloat()**.
- It converts the quantity input into a whole number using **parseInt()**.

Validation and Error Display

Before running calculations, the script checks if the numbers are valid:

- Price Check: If the price is empty, not a number (**isNaN**), or negative, an inline error message appears below the price input.
- Quantity Check: If the quantity is empty, not a number, or less than 1, an inline error message appears below the quantity input.
- If both fields contain clean, valid numbers, any active error messages are hidden immediately.

Live Output Display Update

The selected product name is updated live. If the user leaves the name input blank, it safely falls back to displaying "N/A".

If the input numbers pass validation, the total cost is calculated (Price x Quantity), formatted neatly with two decimal places (e.g., \$25.00), and rendered in the summary area. If validation fails, the total safely defaults to \$0.00.

Source Code

1. index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product-Calculator</title>
  <!-- Css-linking -->
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>

  <div class="card">
    <h2>Product-Calculator</h2>
    <form id="productForm" onsubmit="event.preventDefault();">
      <!-- Product name -->
      <div class="form-group">
        <label for="productName">Product Name:</label>
        <input type="text" id="productName" name="productName" required
placeholder="monitor">
      </div>

      <div class="form-group">
        <label for="productPrice">Unit Price ($)</label>
        <input
          type="number"
          id="productPrice"
          placeholder="Enter price">
      </div>
    </form>
  </div>
</body>
</html>
```

```

        step="0.01"
        min="0"
        value="25.00"
    >
    <div class="error-message" id="priceError">Please enter a valid positive
price.</div>
</div>

    <div class="form-group">
        <label for="productQuantity">Quantity</label>
        <input
            type="number"
            id="productQuantity"
            placeholder="Enter quantity"
            step="1"
            min="1"
            value="1"
        >
        <div class="error-message" id="quantityError">Please enter a valid positive
quantity (min 1).</div>
    </div>
</form>

    <div class="result-container">
        <div class="summary-row">
            <span>Selected Product:</span>
            <span id="displayName" class="product-display-name">Wireless Mouse</span>
        </div>

        <div class="total-row">
            <span>Total Amount:</span>
            <span class="total-price" id="totalPrice">$25.00</span>
        </div>
    </div>
</div>
</body>
</html>

```

2. style.css

```

/* Base Reset & Layout */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
    background-color: #f4f7f6;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    padding: 20px;
}

/* Card Container */
.calculator-card, .card {
    background: #ffffff;
    padding: 30px;
}

```

```
border-radius: 12px;
box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);
width: 100%;
max-width: 450px;
}

h2 {
  text-align: center;
  color: #333333;
  margin-bottom: 24px;
  font-weight: 600;
}

/* Form Styling */
.form-group {
  margin-bottom: 18px;
}

label {
  display: block;
  margin-bottom: 6px;
  color: #555555;
  font-weight: 500;
  font-size: 14px;
}

input {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid #cccccc;
  border-radius: 6px;
  font-size: 15px;
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

input:focus {
  outline: none;
  border-color: #007bff;
  box-shadow: 0 0 6px rgba(0, 123, 255, 0.25);
}

/* Error Message */
.error-message {
  color: #dc3545;
  font-size: 12px;
  margin-top: 4px;
  display: none;
}

/* Output Display Area */
.result-container {
  margin-top: 24px;
  padding-top: 20px;
  border-top: 2px dashed #e0e0e0;
}

.summary-row {
  display: flex;
  justify-content: space-between;
  margin-bottom: 8px;
  font-size: 14px;
}
```

```

    color: #666666;
}

.product-display-name {
    font-weight: 600;
    color: #333333;
}

.total-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-top: 12px;
    font-size: 18px;
    font-weight: bold;
    color: #2c3e50;
}

.total-price {
    color: #28a745;
    font-size: 22px;
}

```

3. script.js

```

// Selecting DOM Elements from HTML page to add functionality to them
const productNameInput = document.getElementById('productName');
const productPriceInput = document.getElementById('productPrice');
const productQuantityInput = document.getElementById('productQuantity');
const displayName = document.getElementById('displayName');
const totalPriceDisplay = document.getElementById('totalPrice');
const priceError = document.getElementById('priceError');
const quantityError = document.getElementById('quantityError');

function calculateTotal() {
    // Parse input values as they are strings by default and float because price can be
    decimal
    const price = parseFloat(productPriceInput.value);
    const quantity = parseInt(productQuantityInput.value, 10);
    const name = productNameInput.value.trim();

    let isValid = true;

    // Validate Price field if price is greater than 0 or not a number or empty
    if (isNaN(price) || price < 0) {
        priceError.style.display = 'block';
        isValid = false;
    } else {
        priceError.style.display = 'none';
    }

    // Validate Quantity field same as price but quantity should be greater than 0
    if (isNaN(quantity) || quantity < 1) {
        quantityError.style.display = 'block';
        isValid = false;
    } else {
        quantityError.style.display = 'none';
    }

    // Update Product Name display dynamically
    displayName.textContent = name !== "" ? name : "N/A";
}

```

```
// Update Total Price display if inputs are valid
if (isValid) {
    const total = price * quantity;
    totalPriceDisplay.textContent = `$$${total.toFixed(2)}`;
} else {
    totalPriceDisplay.textContent = "$0.00";
}
}

// Attach Event Listeners for continuous input changes (triggers without page refresh)
productNameInput.addEventListener('input', calculateTotal);
productPriceInput.addEventListener('input', calculateTotal);
productQuantityInput.addEventListener('input', calculateTotal);

// Initial calculation call when the page loads
calculateTotal();
```

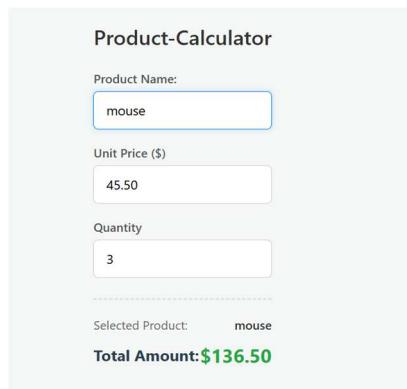
Execution Steps

- Place index.html, style.css, and script.js together inside a single project folder.
- Open index.html directly using a web browser or launch it using Visual Studio Code Live Server.
- Modify the Product Name, Unit Price, and Quantity input values.
- Watch the summary section recalculate instantly without any page refresh.
- Test invalid scenarios (such as negative prices or zero quantity) to see the live error indicators and reset behavior.

Test Cases & Verification Results

TC-01 — Valid Price and Quantity

Input: Price = 45.50, Quantity = 3 Expected: Total = \$136.50 Status: Passed



The screenshot shows a web application titled "Product-Calculator". It has three input fields: "Product Name:" with the value "mouse", "Unit Price (\$)" with the value "45.50", and "Quantity" with the value "3". Below these fields, it says "Selected Product: mouse". At the bottom, it displays "Total Amount: \$136.50" in green text.

TC-02 — Empty Price Field

Input: Price left blank Expected: Price error message shown, Total resets to \$0.00

Status: Passed

The screenshot shows a web form titled "Product-Calculator". It has three input fields: "Product Name" with the value "mouse", "Unit Price (\$)" which is empty and has a red error message "Please enter a valid positive price." below it, and "Quantity" with the value "3". Below the inputs, it shows "Selected Product: mouse" and "Total Amount: \$0.00".

TC-03 — Decimal Quantity

Input: Quantity = 2.5 Expected: Should reject decimal or round properly

Status: Failed — value is silently truncated to 2 with no warning

The screenshot shows the "Product-Calculator" form with "Product Name" as "mouse", "Unit Price (\$)" as "40", and "Quantity" as "2.5". The "Selected Product" is "mouse" and the "Total Amount" is "\$80.00".

TC-04 — Empty Product Name

Input: Product name left blank, valid Price/Quantity entered Expected: Falls back to displaying "N/A"

Status: Passed

The screenshot shows the "Product-Calculator" form with "Product Name" as an empty field, "Unit Price (\$)" as "40", and "Quantity" as "2.5". The "Selected Product" is "N/A" and the "Total Amount" is "\$80.00".

Result / Conclusion

The Interactive Product Calculator application was successfully built and tested using HTML, CSS, and JavaScript. Real-time event handling, dynamic DOM updating, input parsing, validation, and instantaneous total price calculations without refreshing the page all function as intended.